

DSP Project

Complete Setup & Run Guide

Ganesh172919/DSP_Project

Face Authentication with Traditional Pipeline & Vision Language Models

1. Prerequisites — What You Need Before Starting

Install all of the following tools before attempting to run the project.

1.1 Required Software

Tool	Required Version	Notes
Python	3.11 or 3.12	Must be added to system PATH
Node.js	20 or later	Required for frontend
npm	Comes with Node.js	Used to install frontend packages
Docker Desktop	Latest stable	Only needed for Docker method

1.2 Additional Recommendations (Windows)

- Use PowerShell as your terminal
- Allow camera access in your browser for localhost
- Keep at least several GB of free disk space (models and dependencies are large)

2. Download the Project from GitHub

2.1 Repository

GitHub repository: https://github.com/Ganesh172919/DSP_Project

2.2 Download Steps

1. Open the repository link in your browser
2. Click the green Code button
3. Click Download ZIP
4. Wait for the ZIP file to finish downloading

2.3 Extract the ZIP File

5. Go to your Downloads folder
6. Right-click the ZIP file

7. Click "Extract All..."
8. Choose an easy-to-find location, for example:
`C:\Users\YourName\Downloads\DSP Project`

Note: GitHub may name the extracted folder DSP_Project-main. You can keep that name or rename it to DSP Project.

3. Folder Structure

3.1 Required Top-Level Structure

After extraction, the project folder must contain at least:

```
DSP Project/  
|-- .env.example  
|-- backend/  
|-- frontend/  
|-- docs/  
|-- docker-compose.yml  
|-- README.md  
|-- walkthrough.md  
|-- analysis_and_approaches.md  
`-- vlm_approaches.md
```

If backend, frontend, docs, docker-compose.yml, and README.md are present, you can continue.

3.2 Required Backend Structure

```
backend/  
|-- app/  
|   |-- config.py  
|   |-- main.py  
|   |-- pipeline.py  
|   |-- vlm_config.py  
|   |-- vlm_pipeline.py  
|   |-- vlm_routes.py  
|   |-- crypto.py  
|   |-- video_utils.py  
|   |-- db/  
|   `-- models/  
|-- data/  
|-- keys/  
|-- training/  
|-- weights/  
|-- requirements.txt  
|-- vlm_requirements.txt  
`-- Dockerfile
```

3.3 Required Frontend Structure

```
frontend/  
|-- src/  
|   |-- api/  
|   |-- components/  
|   |-- pages/  
|   |-- `-- utils/  
|-- package.json  
|-- package-lock.json  
|-- vite.config.js  
|-- Dockerfile
```

3.4 Final Folder Structure After Successful Setup

After running the project at least once, the complete structure will look like this:

```
DSP Project/  
|-- .env  
|-- .env.example  
|-- backend/  
|   |-- app/  
|   |-- data/  
|   |   |-- auth.db  
|   |   |-- `-- vlm_ref_frames/  
|   |-- keys/  
|   |   |-- private.pem  
|   |   |-- `-- public.pem  
|   |-- training/  
|   |-- venv/ # manual setup only  
|   |-- weights/  
|   |   |-- face_detection_yunet_2023mar.onnx  
|   |   |-- w600k_r50.onnx  
|   |   |-- `-- vlm_cache/  
|   |-- requirements.txt  
|   |-- vlm_requirements.txt  
|   |-- `-- Dockerfile  
|-- frontend/  
|   |-- node_modules/ # manual setup only  
|   |-- src/  
|   |-- package.json  
|   |-- package-lock.json  
|   |-- vite.config.js  
|   |-- `-- Dockerfile  
|-- docs/  
|   |-- BEGINNER_GITHUB_ZIP_TO_RUN_GUIDE.md  
|   |-- FOLDER_STRUCTURE_AND_RUN_GUIDE.md  
|   |-- SETUP_AND_OPERATIONS.md  
|   |-- `-- VLM_HYBRID_AUTHENTICATION.md  
|-- docker-compose.yml
```


5.1 Backend Setup

Open the FIRST PowerShell window. Navigate to the backend folder and run:

```
cd "C:\Users\YourName\Downloads\DSP Project\backend"
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -r vlm_requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

What each command does:

- `python -m venv venv` — creates a Python virtual environment
- `.\venv\Scripts\Activate.ps1` — activates the virtual environment
- `pip install -r requirements.txt` — installs core backend packages
- `pip install -r vlm_requirements.txt` — installs VLM-related packages
- `uvicorn ...` — starts the FastAPI backend on port 8000

When the backend starts successfully, open this URL to confirm:

```
http://localhost:8000/health
```

5.2 What Happens on First Backend Run

On the very first run, the backend automatically creates or downloads:

- `backend/data/auth.db`
- `backend/data/vlm_ref_frames/`
- `backend/keys/private.pem`
- `backend/keys/public.pem`
- `backend/weights/face_detection_yunet_2023mar.onnx`
- `backend/weights/w600k_r50.onnx`

Note: If VLM endpoints are used, VLM model files will also be downloaded into `backend/weights/vlm_cache/`. This first-time download can take a while on slow connections.

5.3 Optional: Warm Up the VLM Model

After the backend starts, you can pre-load the VLM model before the first authentication attempt. Open this URL:

```
http://localhost:8000/api/v1/vlm/status
```

Or send a warmup request from PowerShell:

```
curl.exe -X POST http://localhost:8000/api/v1/vlm/warmup
```

5.4 Frontend Setup

Open a SECOND PowerShell window. Navigate to the frontend folder and run:

```
cd "C:\Users\YourName\Downloads\DSP Project\frontend"
npm install
npm run dev
```

What each command does:

- `npm install` — downloads and installs all frontend JavaScript dependencies
- `npm run dev` — starts the React/Vite development server on port 5173

Once the frontend starts, open:

```
http://localhost:5173
```

5.5 Useful Health Check Commands

```
curl.exe http://localhost:8000/health
curl.exe http://localhost:8000/api/v1/vlm/status
curl.exe -X POST http://localhost:8000/api/v1/vlm/warmup
```

6. Method B — Docker Run (One Command)

Use this method if you want Docker to handle everything for you. Docker will build and run both the backend and frontend in isolated containers.

6.1 Install and Start Docker Desktop

9. Install Docker Desktop from <https://www.docker.com/products/docker-desktop>
10. Open Docker Desktop
11. Wait until Docker shows that it is running (check system tray icon)

6.2 Make Sure .env Exists

From the project root in PowerShell:

```
Copy-Item .env.example .env
```

You only need to do this once.

6.3 Start the Full Stack

From the project root, run:

```
docker compose up --build
```

What this does:

- Builds the backend Docker image
- Builds the frontend Docker image
- Starts both containers together
- Passes all root .env values to the container environment

Once running, open these URLs:

Service	URL
Frontend	http://localhost:5173
Backend	http://localhost:8000
Health Check	http://localhost:8000/health

6.4 Useful Docker Commands

Stop all Docker services when you are done:

```
docker compose down
```

View logs if something is not working:

```
docker compose logs backend  
docker compose logs frontend
```

7. Pages and API Endpoints

7.1 Application Pages

Page	URL
Standard Register	http://localhost:5173/register
Standard Login	http://localhost:5173/login
VLM Register	http://localhost:5173/vlm-register
VLM Hybrid Login	http://localhost:5173/vlm-login
VLM Pure Login	http://localhost:5173/vlm-pure

7.2 API Endpoints

Method	Endpoint	Description
GET	/health	Health check
POST	/api/v1/register	Register a new user (standard)
POST	/api/v1/authenticate/video	Authenticate with video (standard)
POST	/api/v1/vlm/register	Register VLM reference frames
POST	/api/v1/vlm/authenticate	Hybrid VLM authentication
POST	/api/v1/vlm/authenticate/pure	Pure VLM authentication
GET	/api/v1/vlm/status	Check VLM model status
POST	/api/v1/vlm/warmup	Pre-load VLM model

8. Models and Files Loaded During Setup

8.1 Traditional Pipeline Files

- YuNet face detector — face_detection_yunet_2023mar.onnx
- ArcFace recognition model — w600k_r50.onnx
- SQLite database — backend/data/auth.db
- JWT key pair — backend/keys/private.pem and public.pem

8.2 VLM Files

VLM model files are downloaded into:

```
backend/weights/vlm_cache/
```

The model selected depends on available hardware and the VLM_MODEL setting:

VLM_MODEL Value	Model Selected	Best For
auto	Chosen by backend	Default; adapts to hardware
smolvlm	SmoVLM-256M-Instruct	Low-RAM systems
moondream	moondream2	Mid-range systems
qwen	Qwen2.5-VL	Higher-end systems
disabled	None	Skip VLM entirely

Note: On low-RAM systems, the backend may auto-select SmoVLM-256M-Instruct. VLM loading is usually much slower than the traditional pipeline, especially on CPU-only systems.

9. How the VLM Anti-Spoof Protection Works

The VLM prompt is now stricter in both hybrid and pure modes. Access is denied if any authentication frame shows:

- A mobile phone or tablet
- A laptop screen, monitor, or TV
- A replayed video or recorded clip
- A printed photo or hard copy
- A picture, poster, or displayed face
- A face inside another screen, gallery image, playback window, or rectangle
- A face blocked by a device
- A full face that is not clearly visible
- Frozen eye states with weak blink evidence

9.1 Hybrid Mode Extra Safeguard

If the VLM returns any of these explicit spoof red flags, the hybrid route denies immediately:

- `visible_mobile_phone`
- `printed_photo`
- `replayed_video`
- `full_face_not_visible`

10. Recommended First Test Sequence

Follow this order to confirm the complete project is working correctly:

12. Create the `.env` file (Copy-Item `.env.example` `.env`)
13. Start the backend (uvicorn ...)
14. Confirm health: open `http://localhost:8000/health`
15. Start the frontend (npm run dev)
16. Open `http://localhost:5173`
17. Register one user at `/register`
18. Test standard login at `/login`
19. Register VLM reference frames at `/vlm-register`
20. Test hybrid VLM login at `/vlm-login`
21. Test pure VLM login at `/vlm-pure`

This is the safest beginner path to confirm the whole project — backend, frontend, standard pipeline, and VLM pipeline — are all working.

11. Troubleshooting

Problem	Solution
python is not recognized	Install Python 3.11 or 3.12 and reopen PowerShell
npm is not recognized	Install Node.js 20+ and reopen PowerShell
docker compose is not recognized	Install Docker Desktop and make sure it is running
Backend is slow on first VLM request	Model download/loading is happening in the background. Wait for it to complete.
Browser camera does not work	Allow camera permission for localhost in your browser settings
VLM denies access when phone is visible	This is intentional — the stricter anti-spoof rules are working correctly
Backend port 8000 already in use	Stop any other service on port 8000 or change the port in the uvicorn command
Frontend port 5173 already in use	Run npm run dev -- --port 5174 to use a different port

12. Key Runtime Paths

Path	Contents
backend/data/auth.db	SQLite user database
backend/data/vlm_ref_frames/	VLM registration reference frame images
backend/weights/	Traditional pipeline ONNX model files
backend/weights/vlm_cache/	Downloaded VLM model files
backend/keys/	JWT RSA private and public key files